REP-2: Sales Report Upgrade

Instructions for Terry (Part 2, checkout and sales)

Prepared 2026-11-04 from the TA's Checkpoint 3 feedback and the current code. Size L. Owner: Terry.

Item	Details
Your report	Sales report, page **/staff/reports/sales** (managers only). Today it has a date range, a daily table, revenue by category and the top 5 products by units.
What you add	Profit margin, cost of goods sold (COGS), average transaction value, best and worst products, comparison with a previous period, filters for category, product and payment method, and daily, weekly and monthly trends, plus KPI cards, charts and a 'What this means' panel.
Files you create	`db/reports/sales_lines.sql`, `db/reports/test_sales_lines.sql`, `lib/salesMetrics.js`, `lib/salesPeriods.js`, tests under `tests/`.
Files you change	`server/reports/sales.js`, `src/app/pages/reports/SalesReport.jsx` (and a small folder of local components next to it).
Files you must not touch	`db/core/schema.sql` (frozen), `db/core/views.sql` (new views go in your own file), `package.json` (REP-7 installs the chart library), `lib/reportPeriods.js` (REP-7 creates it).
Depends on	Nothing to start. REP-7 provides charts and shared filter components later; REP-0 provides enough demo data for trends. Until then build the data, endpoint, KPI cards and insights, then add charts.
Done when	One sale checked by hand matches revenue minus COGS, switching to weekly or monthly regroups the chart, and the comparison period matches a query run by hand.

Important. SQL in this guide is a hand-checked starting point. It was written against `schema.sql` and follows the style of the existing views, but could not be run here (no SQL Server in the assistant's workspace). Run it on your local SQL Server Express first. Never run anything against the hosted Azure database yourself: the lead loads new SQL there (INT-1).

1. Why this report has to change

The TA's overall comment on all reports was: *"Your reports should go beyond showing what happened and help management understand why it happened, whether performance is improving or declining, and what actions the business should take. Include more meaningful calculations, comparisons, trends, visuals and flexible filters."*

For the Sales report the TA asked for **profit margins, cost of goods sold, average transaction value, best and worst products and comparisons with previous periods**, and for filters by **category, product, payment method** and daily, weekly and monthly trends.

What exists today versus what the TA asked for

TA expectation	Today	What you add
Profit margin and COGS	Only revenue and tax. The report never says whether the store made money.	COGS, gross profit and margin percent as KPIs, by category and by product.
Average transaction value	Not shown. Orders and revenue are in the table, so a reader must divide.	Average order value KPI, with its change, and items per order.
Best and worst products	Top 5 by units only. No worst list.	Best and worst 5 by revenue and by profit, plus products with zero sales in the period.
Comparison with previous periods	None.	Every KPI shows the change against the previous period (or the same period last year). A trend line shows both periods.
Filters: category, product, payment method	Only a date range.	Category, product and payment method filters that change every number on the page.
Daily, weekly, monthly trends	A daily table only.	A group-by switch (day, week, month) that regroups the trend chart and the table.
Explain why and what to do	Management must work it out.	A 'What this means' panel: 'Revenue is up 12%: orders +9%, average order +3%', 'Margin fell from 31% to 27%, mostly in Accessories'.

The point of the change: today a manager sees *how much came in*. After the change a manager sees *whether the business is getting better or worse, what changed it, and what to do*. Everything you add should help answer 'why did this number move?'

2. The data you will use and its limits

Table	Columns you need	Used for
SALES_TRANSACTION	Transaction_id, Customer_id, Transaction_date, Subtotal, Status ('open', 'completed', 'voided')	Only Status = 'completed' counts.
TRANSACTION_ITEM	Transaction_id, Product_id, Quantity, Unit_price, Discount_amount, Line_total (= Quantity x Unit_price - Discount_amount)	Revenue per line after the discount, before tax.
PRODUCT	Product_id, SKU, Name, Cost_price, Category_id	Cost, names, zero-sale list.
CATEGORY	Category_id, Name	Category filter and margin by category.
PAYMENT	Transaction_id, Method ('credit' or 'debit'), Amount	Payment method filter. One payment per sale today.
PRODUCT_RETURN / _ITEM	Return_id, Transaction_id, Status ('requested', 'approved', 'denied', 'refunded'), Product_id, Quantity	Only returns with Status = 'refunded' reduce revenue.

Rule. Every query and every join must filter **Marked_for_deletion = 0**, including the lookup tables. The application login cannot DELETE rows; the project uses soft delete everywhere.

Limits you must respect and print on the page

- **COGS uses today's cost.** `TRANSACTION_ITEM` does not store the cost at the time of sale, and the schema is frozen. COGS = units x `PRODUCT.Cost_price` now. Label it 'COGS at current cost'. If a cost changed, old margins move.
- **Revenue definition.** Revenue = `Line_total`: price after discounts, before tax. This matches the existing views. Tax is never part of revenue or profit.
- **Refunds belong to the original sale day.** The existing views attribute a refund to the day of the sale, not the day of the refund. Keep the same rule so the new report agrees with the old one, and say so on the page.
- **Refunded goods are assumed restocked.** COGS for a refunded unit is removed with its revenue (net COGS = (quantity - refunded quantity) x cost).
- **Payment is card only** (credit or debit), one payment row per sale. A payment method filter therefore has two values.
- **The seed is tiny** (8 sales). Trends and comparisons need REP-0. Every panel needs an empty state ('no sales in this period', 'no data for the comparison period').
- **Category is the product's current category.** If a product is recategorised, history moves with it.

3. New parameters

GET /api/reports/sales keeps working with no parameters (last 30 days, as today). All parameters are optional, validated on the server, echoed back in the response, and kept in the page URL so a filtered view can be shared.

Parameter	Values	Default	Effect and validation
from, to	YYYY-MM-DD	last 30 days	Already validated by `realDate` in `sales.js` (real calendar date, start not after end). Keep that. Also reject ranges longer than 400 days (HTTP 400).
category	A Category_id (integer)	all	Only lines of products in that category. Not an integer: 400.
product	A Product_id (integer)	all	Only lines of that product.
method	credit, debit	all	Only sales paid by that method. Anything else: 400.
groupBy	day, week, month	day	Buckets of the trend and trend table. Weeks start on Monday. If the range is over 92 days and groupBy is day, switch to week and return `groupByUsed` so the page can tell the user.
compare	previous, lastyear, none	previous	previous = the period of equal length immediately before `from`. lastyear = the same dates one year earlier. none = no comparison and no change figures.
limit	1 to 20	5	How many best and worst products to list.

Date presets (this month, last 30 days, this quarter, year to date) are page buttons that simply fill `from` and `to`; the server does not need to know about them.

Coordination. REP-7 will later provide a shared `lib/reportPeriods.js` that every report uses for 'previous period'. Until it merges, put your own version in `lib/salesPeriods.js` with a clear comment. When REP-7 lands, replace your file by a one-line re-export; the function names should therefore match these: `previousPeriod(from, to)`, `lastYearPeriod(from, to)`, `buckets(from, to, groupBy)`.

4. Metrics and formulas

Keep every calculation in pure functions in `lib/salesMetrics.js` (not in `server/`, where every file becomes a Vercel route). Functions take plain rows and return plain values so they can be tested with fixtures.

Metric	Formula	Notes
Gross revenue	SUM(Line_total)	After discounts, before tax, before refunds.
Refunds	SUM(Refunded_revenue)	Refunded lines only, at the original sale day.
Net revenue	Gross revenue - Refunds	The headline revenue KPI. Same as the Net_revenue column of VW_SALES_REPORT.
COGS	SUM((Quantity - Refunded_qty) x Cost_price)	At current cost.
Gross profit	Net revenue - COGS	Can be negative: show it in words as a loss.
Margin percent	Gross profit / Net revenue x 100	Null when net revenue is 0. One decimal place. Changes are shown in percentage points, not percent.
Orders	COUNT(DISTINCT Transaction_id)	With a category or product filter: orders that contain at least one matching line. Say so on the page.
Average order value	Net revenue / Orders	The TA's 'average transaction value'. Null when Orders is 0.
Units	SUM(Quantity)	Gross units.
Items per order	Units / Orders	
Discount given	SUM(Discount_amount)	Plus discount rate = Discount given / (Gross revenue + Discount given).
Refund rate	Refunds / Gross revenue	
Change (percent)	(current - previous) / previous x 100	Null when previous is 0 (show 'new', not infinity). Null when compare = none.
Change (points)	current margin - previous margin	For margin, discount rate and refund rate.
Revenue explained	Orders change and average order change: (1 + o) x (1 + a) - 1 = revenue change	Used by the 'why' sentence. For 12% revenue growth with 9% more orders, the average order grew about 2.8%.
Profit per product	Net revenue of the product - COGS of the product	Used for best and worst lists. Rank ties by name so the order is stable.
Zero-sale products	Active products matching the filters with no completed line in the period	Come from PRODUCT, not from the view (they have no lines). List up to the limit, by stock value descending.

Bucketing the trend

- Ask SQL for **one row per day** (grouped by `Sale_date`) and do the weekly and monthly grouping in JavaScript in `lib/salesPeriods.js`. SQL week numbers depend on the server's first-day-of-week setting and give different results in Azure and on a laptop.
- Weeks start on Monday and are labelled by their Monday date. Months are labelled 'Oct 2026'.
- Fill empty buckets with zero, so a day with no sales appears as a zero on the chart instead of a gap.
- Align the previous period to the current one **by position** (day 1 against day 1, week 1 against week 1) so both lines can be drawn on the same axis. Return the previous dates too, so tooltips can show them.

5. Build procedure

Step 1. Create the line-level view (db/reports/sales_lines.sql)

A view cannot take parameters, so the view returns one row per sold item and the endpoint filters and groups. The refund formula is the one already used in `VW_SALES_REPORT`, applied per line.

```
-- VW_SALES_LINES: one row per item of a completed sale (not flagged)
CREATE OR ALTER VIEW VW_SALES_LINES AS
SELECT s.Transaction_id,
       CAST(s.Transaction_date AS DATE) AS Sale_date,
       s.Customer_id,
       ti.Product_id, p.SKU, p.Name AS Product_name,
       c.Category_id, c.Name AS Category_name,
       pay.Method AS Payment_method,
       ti.Quantity, ti.Unit_price, ti.Discount_amount,
       CAST(ti.Quantity * ti.Unit_price AS DECIMAL(12,2)) AS Price_before_discount,
       CAST(ti.Line_total AS DECIMAL(12,2))               AS Revenue,
       f.Refunded_qty,
       CAST(ROUND(ti.Line_total * f.Refunded_qty / ti.Quantity, 2) AS DECIMAL(12,2)) AS Refunded_revenue,
       CAST((ti.Quantity - f.Refunded_qty) * p.Cost_price AS DECIMAL(12,2))          AS Cost
FROM   SALES_TRANSACTION s
JOIN   TRANSACTION_ITEM ti ON ti.Transaction_id = s.Transaction_id AND ti.Marked_for_deletion = 0
JOIN   PRODUCT p  ON p.Product_id  = ti.Product_id  AND p.Marked_for_deletion = 0
JOIN   CATEGORY c ON c.Category_id = p.Category_id  AND c.Marked_for_deletion = 0
OUTER APPLY (SELECT TOP (1) py.Method
             FROM   PAYMENT py
             WHERE  py.Transaction_id = s.Transaction_id AND py.Marked_for_deletion = 0
             ORDER BY py.Paid_at, py.Payment_id) pay
CROSS APPLY (SELECT ISNULL(SUM(ri.Quantity), 0) AS Refunded_qty
             FROM   PRODUCT_RETURN r
             JOIN   PRODUCT_RETURN_ITEM ri ON ri.Return_id = r.Return_id
             WHERE  r.Transaction_id = ti.Transaction_id
               AND  ri.Product_id    = ti.Product_id
               AND  r.Status = 'refunded'
               AND  r.Marked_for_deletion  = 0
               AND  ri.Marked_for_deletion = 0) f
WHERE  s.Status = 'completed'
  AND  s.Marked_for_deletion = 0;
GO
```

Check: run the file twice (it must not fail the second time). For one completed sale, compare `SUM(Revenue)` over its lines with `SALES_TRANSACTION.Subtotal` and compare the total of `Revenue - Refunded_revenue` by day with `Net_revenue` from `VW_SALES_REPORT`. They must agree to the cent.

> **Trap.** Do not join `PAYMENT` as a plain JOIN. If a sale ever had two payment rows, the lines would be counted twice and revenue would double. The OUTER APPLY with TOP (1) cannot multiply rows.

Step 2. Write a rolled-back test script (db/reports/test_sales_lines.sql)

Follow the pattern of `verify.sql`: open a transaction, insert known rows, assert the view, then ROLLBACK so nothing is left behind. Cover at least:

- A completed sale with two lines (one with a discount): Revenue values, Price_before_discount, Cost = quantity x cost.
- A partial refund (refunded return of 1 of 2 units): Refunded_qty = 1, Refunded_revenue = half the line, Cost counts one unit only.
- A return that is only 'requested' or 'denied': no refund counted.
- An 'open' or 'voided' sale and a flagged line or flagged product: not in the view.
- A sale without a payment row: still appears, with a NULL Payment_method.

Step 3. Write the pure metric and period modules

Function	Input	Output
previousPeriod(from, to)	Two date strings	{ from, to } of equal length immediately before. Example: 2026-10-05 to 2026-11-03 (30 days) gives 2026-09-05 to 2026-10-04.
lastYearPeriod(from, to)	Two date strings	Same dates one year earlier. 29 February maps to 28 February.
buckets(from, to, groupBy)	Dates, 'day', 'week' or 'month'	Ordered list of { key, label, start, end } with no gaps.
summarize(rows)	Rows of the view (already filtered)	{ grossRevenue, refunds, netRevenue, cogs, grossProfit, marginPct, orders, avgOrder, units, itemsPerOrder, discountGiven, discountRatePct, refundRatePct }.
compareKpis(current, previous)	Two summaries	Same keys, each { value, previous, change, changeKind }, changeKind being 'percent', 'points' or 'new'.
trendSeries(dayRows, buckets)	Daily aggregates and buckets	Arrays aligned to the buckets: revenue, profit, orders, marginPct.
rankProducts(rows, limit)	Rows of the view	{ bestByRevenue, worstByRevenue, bestByProfit, worstByProfit }.
marginByCategory(rows, prevRows)	Both periods	Per category: revenue, profit, marginPct, previousMarginPct, shareOfRevenue.
buildSalesInsights(current, previous, categories, products)	Summaries and lists	Up to 5 { id, severity, text, action }. See section 7.

Use integer cents when summing money (the way `server/reports/inventory.js` already does) so totals never drift by a cent.

Step 4. Extend the endpoint

1. In `server/reports/sales.js` keep the three existing queries and the keys **from, to, days, byCategory, topProducts** exactly as they are, so the old screen and any test that reads them keep working. They ignore the new filters; say so in a comment.

2. Validate the new parameters (section 3). Check `lib/validate.js` for an integer helper (the rewards report uses `validate.integer`); for the lists of allowed values throw `new HttpError(400, ...)`.
3. Work out the comparison dates with `salesPeriods`.
4. Read the lines of both periods with **one parameterised query each** against `VW_SALES_LINES`. Add `AND Category_id = @category` only when the filter is present; always pass values as named parameters.
5. Get the daily aggregates with `GROUP BY Sale_date` and let the library do the bucketing.
6. Return the new keys: **filters, groupByUsed, period, comparePeriod, kpis, trend, categories, best, worst, zeroSale, byMethod, insights**.
7. No new route file is needed (same endpoint), so `npm run routes` should report no change. Run it anyway; `npm test` fails if the generated list is stale.

> **Keep.** The endpoint stays manager-only, exactly as today.

Step 5. Rebuild the page (SalesReport.jsx)

1. Put new small components in `src/app/pages/reports/sales/` (filter bar, KPI card, trend, product lists, insights). When REP-7 merges you replace them with the shared ones; keeping them local avoids conflicts now.
2. Layout, top to bottom: page header, filter bar, KPI cards, 'What this means' panel, trend chart, category and product sections, daily table. The wireframe on the last page shows it.
3. Filter bar: date presets and two date fields, a compare switch (previous period, same period last year, none), category, product (a searchable list, not a thousand-item select), payment method, group by.
4. Read the filters from the URL query string and write them back when they change.
5. Every KPI card shows its value, the change in words and colour ('up 12% on the previous period'), and the previous value in small text. A lower refund rate is good, so colour by meaning, not by sign.
6. Charts: until REP-7 adds a chart library, draw bars with plain `div` elements and inline styles (allowed by the site's security policy); a trend can be an inline SVG polyline. Swap to the shared components later.
7. Tables are sortable. Keep a daily or weekly table (it is what the old page had) under the charts, and add a CSV download if time allows.
8. Empty states: no sales in the period; no data in the comparison period; no product matches the filter.

Step 6. Tests

Test file	What it checks	How
tests/lib/salesMetrics.test.js	Every formula of section 4 with fixtures: margin with a refund, zero orders, previous period 0 (change = 'new'), negative profit, cents rounding, ranking ties, insight rules and their order.	Plain fixtures, no database.
tests/lib/salesPeriods.test.js	Previous period length and dates, last year on 29 February, week buckets start Monday, month buckets across a year end, no gaps, day range over 92 days.	Plain fixtures.
tests/salesReport.test.js	Bad category, product, method, groupBy or compare gives 400; valid filters reach the query as named parameters; old keys still present; support 403; no login 401; the SQL text contains Marked_for_deletion = 0.	Use `tests/helpers_manager.js` with `jest.mock('../lib/db', ...)` as in `tests/catalog.test.js`.

Do not use `helpers_backoffice.js` for filter tests: its request builder does not support query parameters. It is fine for one smoke test against the real local database.

6. Visual KPIs and chart ideas

The reader first sees the cards, then the meaning panel, then the evidence in charts. Each item says what question it answers; cut anything that does not answer one.

6.1 KPI cards

Card	Value	Change shown	Question it answers
Net revenue	After discounts and refunds, before tax	Percent change	Are we selling more or less?
Gross profit	Net revenue - COGS (at current cost)	Percent change	Are we making money on those sales?
Margin	Profit / net revenue	Points up or down	Are we selling at healthier or thinner margins?
Orders	Completed orders	Percent change	Are more customers buying?
Average order value	Net revenue / orders	Percent change	Do customers spend more per visit?
Refund rate	Refunds / gross revenue	Points up or down (lower is better)	Is something wrong with quality or listings?
Optional: units, discount rate, items per order	See section 4	As above	Where does the money go?

6.2 Charts

Chart	Type	What it shows
Revenue and profit trend	Line, current period solid, previous dashed; follows the group-by switch	Whether performance is improving or declining. The most important chart: the TA's wording.
Profit waterfall	Bridge: price before discounts, minus discounts, minus refunds, minus COGS, equals gross profit	Where the money goes between list price and profit. Explains margin.
Revenue bridge versus previous period	Bridge: previous revenue, plus effect of more orders, plus effect of higher average order, equals current revenue	Why revenue moved (growth from traffic or from basket size).
Margin by category	Grouped bars: this period against previous, with revenue share in the label	Which category is dragging margin down.
Best and worst products	Horizontal bars by profit; the worst are red and can be negative	Which products to push and which to rethink.
Payment method split	Donut or two bars: credit against debit, by revenue and by average ticket	Whether one method brings larger sales.
Weekday pattern	Bars: average revenue per weekday	Which days are strong. Extra if time allows.
Margin trend	Line of margin percent per bucket	Whether discounting is eroding margin over time.

Minimum for 'done': the revenue and profit trend, the margin by category chart and the best and worst products list. The waterfalls are the best 'why' visuals if you can fit them.

7. 'What this means' panel: the insight rules

These are rule-based sentences, not guesses. Each rule has a condition, a sentence built from real numbers and a recommended action. Show at most five, most severe first. Put the rules in `buildSalesInsights` so they can be tested. Skip every comparison rule when compare is none or the previous period has no sales.

Id	Severity	Condition	Sentence and action
revenue	Info or Good or Warning	Revenue change of 5% or more up or down	"Revenue is up 12%: orders +9%, average order +3%." Action: if down, check which category fell (see below).
marginDrop	Danger	Margin fell by 2 points or more	"Margin fell from 31% to 27%, mostly in Accessories." Name the category with the largest margin drop among categories with at least 10% of revenue. Action: review its discounts and costs.
discount	Warning	Discount rate rose by 2 points or more	"Discounts took 9% of list price, up from 5%." Name the discount if only one drove it. Action: check whether the extra sales covered the lost margin.
loss	Danger	A product with negative profit and at least 3 units sold	"Product X lost $Y in this period." Action: its cost is above its selling price after discounts.
refunds	Warning	Refund rate of 5% or more, or up 2 points	"Refunds were 6% of revenue." Name the product with the most refunded units. Action: open the returns report.

Id	Severity	Condition	Sentence and action
basket	Good or Warning	Average order change of 5% or more	"Customers spend $4 more per order." Or less: "The average order fell 7%." Action: bundles or accessories at checkout.
star	Good	One product gives 20% or more of gross profit	"Product X gives 24% of profit." Action: keep it in stock (open the inventory report).
idle	Info	Zero-sale products exist	"N active products had no sales in this period." Action: promote, bundle or lower the reorder level.
allgood	Good	No danger or warning	"Revenue and margin are stable compared with the previous period."

8. Traps to avoid

- **Double counting.** A plain join to PAYMENT or to returns multiplies lines. Use the OUTER APPLY and CROSS APPLY forms shown.
- **Wrong order count with filters.** Count distinct transactions, not lines.
- **Soft delete.** A forgotten `Marked_for_deletion = 0` on a joined table brings deleted products back. Test it.
- **Dates.** Filter with `Sale_date BETWEEN @from AND @to` on the view's date column. Never compare a DATETIME2 to a date string; the existing code converts the end date with CAST for this reason.
- **Time zone.** 'Today' is the server date. On Azure that is UTC, so late in the evening local time the last day can look empty. Acceptable; note it in the PR.
- **Percent of zero.** A previous value of 0 gives 'new', never infinity or NaN.
- **Margin is points.** '31% to 27%' is a 4-point drop, not a 13% drop. Say 'points'.
- **Colour alone.** Pair red or green with words and arrows ('up', 'down').
- **Server files become routes.** Put helper code in `lib/`, not in `server/`.
- **Do not remove the old keys** in this task; a cleanup can come later.

9. Definition of done

[ ]	`db/reports/sales_lines.sql` runs twice without errors on a local database, and every join filters Marked_for_deletion = 0.
[ ]	The rolled-back test script passes and leaves no rows behind.
[ ]	One real sale checked by hand: revenue minus COGS equals the profit shown, and its net revenue agrees with VW_SALES_REPORT.
[ ]	Pure functions in `lib/` have tests for every formula, including refunds, zero orders and previous period 0.
[ ]	The endpoint still returns from, to, days, byCategory and topProducts unchanged, plus the new keys. Every parameter is validated and tested.
[ ]	Switching group by to week or month regroups the chart and the table; the comparison period matches a query you ran by hand.
[ ]	The page has: filter bar, six KPI cards with changes, meaning panel with 3 to 5 insights, at least the trend chart, margin by category and best and worst products, sortable table, empty states.

[ ]	`npm test` passes (run `npm run routes` first if you added any file under `server/`).
[ ]	The page states its limits: COGS at current cost, refunds at the original sale day, orders with a filter mean orders containing matching items.
[ ]	The PR description lists the new parameters, the new files and the manual checks you did.

Questions to ask the lead before you start: (1) Is the default comparison 'previous period' acceptable? Suggested: yes. (2) Should the headline revenue be net of refunds (suggested) or gross? (3) Should zero-sale products include paused ones? Suggested: no.

Wireframe: how the upgraded Sales report page could look

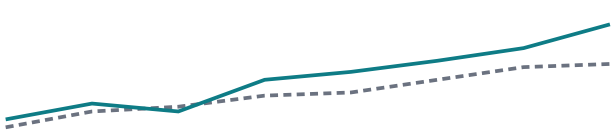
Sales report

Oct 5 to Nov 3, 2026, compared with Sep 5 to Oct 4 (previous period)

Dates: Last 30 days | Compare: Previous period | Category: All | Product: All | Payment: All | Group by: Week

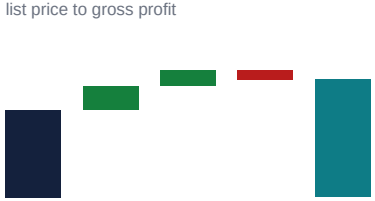
$61,400	$16,600	27.0%	412	$149	6.1%
Net revenue	Gross profit	Margin	Orders	Average order value	Refund rate
up 12% on previous period	up 3%	down 4.0 pts (was 31.0%)	up 9%	up 3%	up 2.0 pts (worse)

Revenue and profit trend
weekly; dashed = previous period

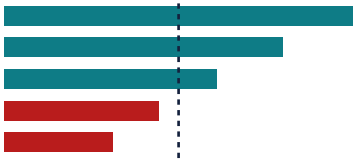


Margin by category
this period vs previous

Profit waterfall
list price to gross profit



Best and worst products
by profit; red = losing money



Payment method split
credit vs debit, by revenue

What this means

- Margin fell from 31% to 27%, mostly in Accessories after the 20% discount. *Review that discount and its costs.*
- Revenue is up 12%: orders +9%, average order +3%. *Keep the campaign running.*
- Refunds were 6% of revenue, up 2 points. *Product X had the most: open returns.*
- Discounts took 9% of list price, up from 5%. *Check the extra sales covered it.*
- 14 active products had no sales. *Promote, bundle or reduce reorders.*

Period	Orders	Units	Net revenue	COGS	Gross profit	Margin	Avg order	Refunds

Reading order: filters, six KPI cards with changes, the 'What this means' panel (right), charts (left), then the table that follows the group-by switch (day, week or month). The numbers are examples only. Every chart box has a one-line caption.

Colour is never alone: a lower refund rate is good, so the colour follows meaning, and the words ('worse', 'up', 'down') are always printed. Until REP-7 installs a chart library, draw bars with plain div elements and the trend as an inline SVG line.